Lesson 14 Teacher's Key: Raising the Stakes

Godot Engine - Instructor Reference Only

VIDEO GAME CREATION

What This Lesson Teaches

Same idea as every engine this week - replace hardcoded literals with wave-dependent formulas - expressed as GDScript, split across Main's spawn_enemies() (introduced Lesson 10) and GameManager's own script, exactly where this course has always kept the enemy-creating code and the shared game state, respectively.

Level 1 - Full Worked Solution

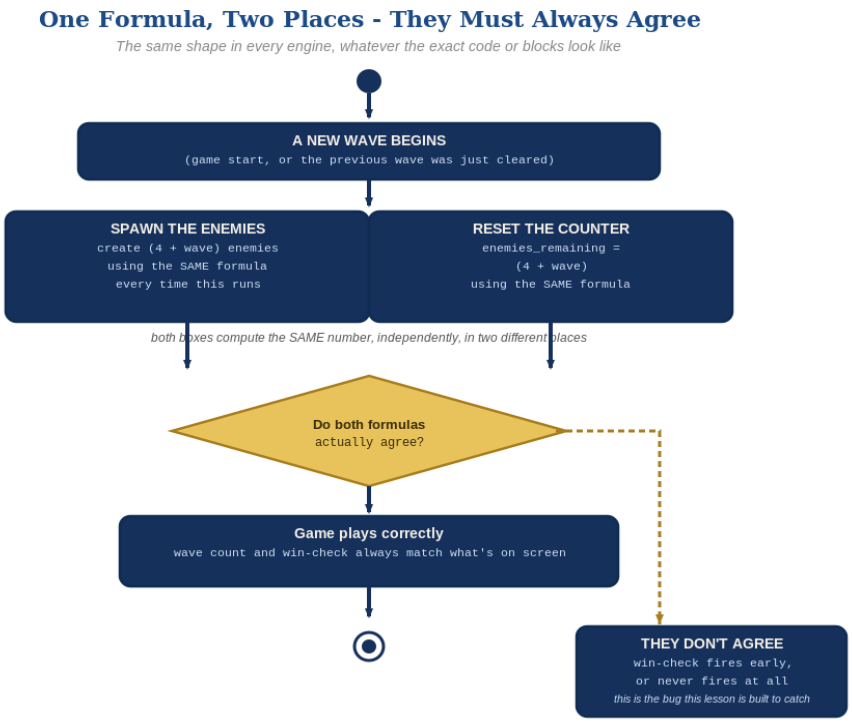
- range(4 + $GameManager.wave) inside spawn_enemies() and enemies_remaining = 4 + wave inside GameManager's two reset spots must all three agree - any mismatch is exactly the kind of bug this Key's troubleshooting section calls out.
- No new variables are needed for Level 1 - this is purely a formula change inside functions that already exist.
- Wave 1 still spawns exactly 5 (4 + 1 = 5), so nothing about the game's opening feel changes - only later waves escalate.

Highlighted lines are new this week; everything else is code you already have.

```
# Main scene's script - spawn_enemies() (revised):
func spawn_enemies():
    for enemy_count in range(4 + $GameManager.wave):
        var new_enemy := enemy_scene.instantiate()
        new_enemy.position = Vector2(150 + enemy_count * 200, 50)
        add_child(new_enemy)
        new_enemy.destroyed.connect($GameManager.on_enemy_destroyed)
        new_enemy.escaped.connect($GameManager.on_enemy_escaped)

# GameManager's script - start_game() (revised, enemies_remaining line
only):
enemies_remaining = 4 + wave

# GameManager's script - on_enemy_destroyed()'s wave-advance else branch
(revised, enemies_remaining line only):
enemies_remaining = 4 + wave
```



Whether the spawn loop and the reset live in two places or two genuinely different scripts, the rule is the same: one formula, referenced everywhere.

Why This Structure

- Keeping the loop bound's wave-dependence inside spawn_enemies() (on Main, via $GameManager.wave) while enemies_remaining's reset stays on GameManager itself is exactly Rule 13's existing split - spawn_enemies() is about creating enemies, enemies_remaining is shared game state, and today's change does not blur that line even though both numbers now depend on the same wave.
- For Level 2, setting enemy.speed on each freshly-instantiated enemy (rather than changing enemy.tscn's own default) keeps the scaling logic entirely inside spawn_enemies() - the enemy scene itself stays exactly as simple as Lesson 7 left it.

Level 2 / Level 3 Talking Points

- Ask students to trace through spawn_enemies() for wave 3 by hand: range(4+3)=range(7) - then separately confirm GameManager's enemies_remaining reset computes the same 7 - a good comprehension check for reading a formula split across two scripts.
- For students comparing engines: point out that Unreal's identical (4 + Wave) formula, expressed as Blueprint math nodes instead of GDScript, is conceptually the exact same idea - a good cross-engine parallel.

Level 2 - complete code:

Highlighted lines are the Level 2 additions on top of the Level 1 code above.

```
# Main scene's script - spawn_enemies() (Level 2 addition, on top of the
Level 1 code above - speed set on each new instance):
func spawn_enemies():
    for enemy_count in range(4 + $GameManager.wave):
        var new_enemy := enemy_scene.instantiate()
        new_enemy.position = Vector2(150 + enemy_count * 200, 50)
        new_enemy.speed = 60.0 + ($GameManager.wave - 1) * 20.0
        add_child(new_enemy)
        new_enemy.destroyed.connect($GameManager.on_enemy_destroyed)
        new_enemy.escaped.connect($GameManager.on_enemy_escaped)

# enemy.tscn's own _process(delta) movement is unchanged - it already
reads its own speed variable from Lesson 7, so no code changes there.
```

Level 3 - no single combined code example:

This week's Level 3 is a "choose ONE" menu of five independent stretch features (destructible shields, a power-up, a bonus enemy, formation-marching movement, or a title/game-over screen) rather than the usual set of small, combinable stretch goals - so a single "all goals combined" code block doesn't fit Rule 14's usual model, and isn't provided here. Full worked solutions for all five options, across the active engines, are planned for the Lesson 15 wrap-up "Complete Codebase" send-home materials instead. Use the Talking Points above and the Handout's Level 3 menu to guide students toward whichever option they choose.

Common Student Mistakes

- Updating spawn_enemies()'s range() but forgetting to update one (or both) of GameManager's enemies_remaining reset spots - or vice versa.
- Forgetting the $GameManager prefix on wave inside spawn_enemies() (it lives on Main, not GameManager) while correctly leaving it off inside GameManager's own start_game()/on_enemy_destroyed() - easy to flip by habit.
- In start_game(), leaving wave = 1 AFTER enemies_remaining's reset instead of before it, so a restart computes enemies_remaining from the previous game's leftover wave number instead of a fresh wave 1.
- Setting enemy.speed BEFORE add_child() instead of after - harmless in Godot 4 either way, but worth confirming the assignment target is actually the enemy just created, not a stale reference.
- Leaving enemy.tscn's default speed := 60.0 as the ONLY place speed is set, forgetting the new per-instance override in spawn_enemies() entirely.